

# INSTRUCTIVO TÉCNICO: ARQUITECTURA DATA LAKEHOUSE

Entorno Profesional: Airflow + PySpark + MinIO (S3)

Autor: Lauro IT | Fecha: Mayo 2026

## 1. Visión General de la Arquitectura

Este ecosistema migra un pipeline de datos local a un entorno de contenedores Docker robusto, implementando las capas **Bronze** (Datos Crudos) y **Silver** (Datos Transformados) bajo el concepto de Data Lakehouse.

- **Orquestación:** Apache Airflow para la automatización de tareas.
- **Procesamiento:** PySpark (Spark 3.5.0) sobre infraestructura Linux.
- **Storage (S3):** MinIO para persistencia de objetos en formato Parquet.
- **Conectividad:** Protocolo S3A para integración nativa Spark-S3.

## 2. Configuración de Red y Conectividad

Un pilar fundamental para el éxito del entorno es la resolución de nombres dentro de la red interna de Docker:

| Entorno             | Endpoint de Conectividad | Motivo                                                 |
|---------------------|--------------------------|--------------------------------------------------------|
| Local (Host)        | http://localhost:9000    | Acceso desde el navegador o herramientas externas.     |
| Contenedor (Docker) | http://minio:9000        | Comunicación interna entre Airflow/Spark y el storage. |

## 3. Lógica de Transformación y Negocio

### A. Resolución de Ambigüedades

Al unir fuentes de datos diversas (CSV y JSON), se implementó una estrategia de limpieza para evitar errores de columnas duplicadas:

```
# Renombrado preventivo para evitar colisión en el Join
df_creditos = df_creditos.withColumnRenamed("MONTO_SOLICITADO", "MONTO_CREDITO_JSON")
```

## B. Ingeniería de Atributos (Scoring de Riesgo)

Se desarrolló una lógica de clasificación basada en reglas de negocio para los perfiles de riesgo:

- **Score Final:** Promedio aritmético entre el scoring interno y el externo (Veraz).
- **Categorización:**
  - Bajo: Score > 700
  - Medio: 400 <= Score <= 700
  - Alto: Score < 400

## 4. Gestión de Dependencias y JARs

Para la integración con S3, es crítico el uso de los siguientes paquetes Maven:

```
spark.jars.packages:  
- org.apache.hadoop:hadoop-aws:3.3.4  
- com.amazonaws:aws-java-sdk-bundle:1.12.262
```

*Nota técnica:* Durante el primer arranque tras un "down", Spark descargará estos archivos (aprox. 160MB), lo que puede retrasar la primera ejecución.

## 5. Troubleshooting y Mantenimiento

**Error 403 / Signature Error en Airflow:** Se identificó como un problema visual de sincronización de la UI que no afecta la lógica de procesamiento si el log final indica SUCCESS.

**Comandos de Salud del Sistema:**

```
# Monitoreo de recursos y red  
docker stats airflow-scheduler  
  
# Seguimiento de ejecución en tiempo real  
docker logs -f airflow-scheduler
```

## 6. Roadmap: Próxima Fase (Capa Gold)

La evolución del sistema contempla la persistencia en bases de datos relacionales:

- **Conexión JDBC:** Integración con Oracle y MySQL para persistir resultados.
- **Heterogeneous Services:** Simulación de entornos híbridos mediante DBLinks para consultas SQL avanzadas.
- **Analista Funcional (AFU):** Desarrollo de reportes directos sobre la Capa Gold.

